# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

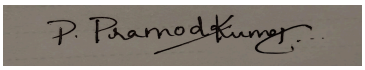
### Department of Computer Science and Engineering

## CO2 ASSESSMENT TOOL 2 – ARCHITECTURE DESIGN ASSIGNMENT

| Course Code | CSA10 / CSA1021 | Course Title | Software Engineering |
|---|---|---|---|
| CO Assessed | CO2 | Assessment | Assessment Tool 2 – Architecture Design Assignment |
| Weightage | 20% | Total Marks | 25 |
| CO2 Statement | Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills | | |
| Student Name | P. Pramod Kumar | Reg. No. | 1925260245 |
| Date | 30-10-2026 | Duration | 60 minutes |
| Slot | A | | |

## Code of Conduct

I, P. Pramod Kumar (1925260245), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Annexure A – Architecture Design Assignment

## Annexure A – Architecture Design Assignment

Instructions to Students: Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. The submission should demonstrate the ability to select, justify, and evaluate a suitable architectural style.

## Questions

1. Analyze System Requirements: Study the given problem statement, identify system objectives, analyze functional and non-functional requirements, and identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture: Select an appropriate architectural style (for example, layered, client-server, MVC, microservices, event-driven, service-oriented, serverless, or hybrid) and justify why it is suitable for the proposed system.

3. Draw the Software Architecture Diagram: Design a clear architecture diagram showing major layers/components, databases, external systems, APIs, and communication mechanisms, using suitable labels and notation.

4. Explain Components and Their Interactions: Describe the purpose and responsibilities of each architectural component, explain how components communicate and exchange data, and illustrate the overall workflow.

5. Discuss Quality Attributes: Evaluate how the proposed architecture addresses scalability, security, performance, reliability, maintainability, and availability.

6. Justify Architectural Decisions: Explain the rationale for architectural choices, discuss trade-offs, and show how the architecture supports future enhancements, integration, and long-term maintainability.

## Rubrics for Evaluation

| Evaluation Criteria | Excellent (4) | Good (3) | Satisfactory (2) | Needs Improvement (1) |
|---|---|---|---|---|
| System Requirement Analysis (30%) | Thoroughly identifies functional/non-functional requirements and explains quality attributes. | Analyzes most requirements with minor omissions. | Identifies basic requirements with partial analysis. | Inaccurate or unclear analysis. |
| Selection of Software Architecture (25%) | Selects the most suitable style with strong technical justification. | Selects a suitable architecture with reasonable justification. | Partially suitable architecture with limited justification. | Inappropriate architecture or no justification. |
| Architecture Diagram and Component Design (20%) | Complete diagram with major components, interfaces, and interactions. | Mostly complete with minor omissions. | Essential components only; limited detail. | Incomplete or technically incorrect. |
| Component Interaction and Quality Attributes (15%) | Clearly explains interactions and all six quality attributes. | Explains most components and attributes. | Limited interaction and attribute explanation. | Poor or missing explanation. |
| Architectural Justification and Technical Presentation (10%) | Strong rationale, trade-offs, future scope, and professional presentation. | Adequate justification with minor gaps. | Limited trade-off or future-scope discussion. | Weak justification and poor organization. |

## Criteria and Marks

| Criteria | Marks |
|---|---|
| System Requirement Analysis (30%) | /5 |
| Selection of Software Architecture (25%) | /5 |
| Architecture Diagram and Component Design (20%) | /5 |
| Component Interaction and Quality Attribute Analysis (15%) | /5 |
| Architectural Justification and Technical Presentation (10%) | /5 |
| TOTAL | /25 |

## Industry Context

Job markets are moving away from traditional four-year degrees toward continuous skills development and micro-credentials. However, job applicants often struggle to compile, share, and verify diverse certifications earned from online courses, bootcamps, and workplace training programs. Employers face slow, expensive verification steps when hiring. The modern educational ecosystem needs a secure, lifelong learning registry where individuals can verify their skills instantly.

## Problem Statement

Design and develop a decentralized Lifelong Learning Ledger that allows educational institutions, employers, and online platforms to issue unalterable micro-credentials. The platform must manage a secure registry of skill badges, course completions, and professional certifications. It must allow students to aggregate these milestones into a digital portfolio and share them with recruiters for instant verification without needing a central school registrar.

## Objectives

1. Build a decentralized platform to issue, claim, and organize digital skill credentials.
2. Implement cryptographic validation layers to prevent document tampering or forgery.
3. Create a secure sharing-control mechanism that lets users manage who sees specific credentials.
4. Integrate with HR hiring tools to automate skill verification during job applications.
5. Support flexible micro-credential updates as technologies and job roles evolve over time.

## Expected Outcomes

1. Fast, low-cost verification of professional skills for hiring managers and recruiters.
2. Better career mobility for professionals through a verified portfolio of lifelong skills.
3. Reduced administrative costs for schools and training academies via automated issuing.
4. A clearer, data-driven understanding of global workforce skills and education trends.

# 1. Introduction

Continuous learning has become an important part of modern education and employment. Individuals gain skills through universities, online learning platforms, bootcamps, professional training, and workplace experience. These learning achievements are often stored in separate systems and issued in different formats.

The proposed Decentralized Lifelong Learning Ledger provides a common digital system for issuing, storing, organizing, sharing, and verifying micro-credentials. A micro-credential is a focused record of a particular skill, course completion, or professional achievement.

The platform connects credential issuers, learners, and employers. Issuers create digitally signed credentials; learners collect them in a personal portfolio; and recruiters verify authenticity through a controlled sharing link or QR-based verification.

The design uses a hybrid architecture: layered application services, independently deployable modules, and event-driven integration. A permissioned ledger or tamper-evident registry stores credential fingerprints and status events, while encrypted off-chain storage holds detailed credential documents and personal information.

The ledger is not intended to publish private learner data. It stores only the minimum proof needed for integrity and status verification. The learner remains in control of sharing access.

# 2. Analyze System Requirements

## 2.1 System objectives

The primary objective is to create a trusted lifelong learning record that can be used across educational institutions, training providers, employers, and online platforms. The system should reduce manual verification, improve portability of credentials, and give learners control over their digital portfolios.

Specific objectives are to support credential issuance, learner claiming, secure storage, portfolio organization, selective sharing, instant verification, revocation/status updates, and integration with external HR and learning systems.

## 2.2 Functional requirements

| ID | Functional requirement | Description |
|---|---|---|
| FR1 | User registration | Register learners, issuers, recruiters, and administrators with verified identities. |
| FR2 | Issuer onboarding | Approve educational institutions, employers, and training platforms as trusted issuers. |
| FR3 | Credential issuance | Create a digitally signed credential with learner ID, issuer, skill, evidence, date, and expiry where applicable. |
| FR4 | Credential claiming | Allow a learner to accept and add an issued credential to their portfolio. |
| FR5 | Credential storage | Store credential metadata securely and protect supporting documents. |
| FR6 | Portfolio management | Group credentials by skill, domain, provider, date, or proficiency. |
| FR7 | Selective sharing | Generate time-limited links or QR codes with learner-approved access scopes. |
| FR8 | Credential verification | Verify issuer signature, credential hash, status, and expiry. |
| FR9 | Revocation/status | Allow authorized issuers to revoke or update credential status with an audit record. |
| FR10 | HR/LMS integration | Expose secured APIs for hiring systems and learning platforms. |
| FR11 | Notifications | Notify learners about new credentials, sharing activity, expiry, and status changes. |
| FR12 | Audit and reporting | Record issuance, access, verification, and administrative actions. |

## 2.3 Requirement interpretation

The most architecture-sensitive functions are credential issuance, cryptographic verification, selective sharing, and integration. Issuance must establish who created a credential. Verification must confirm that the credential has not been altered and remains valid. Sharing must reveal only the information authorized by the learner.

# 3. Non-Functional Requirements and Constraints

## 3.1 Non-functional requirements

| Attribute | Requirement and design approach |
|---|---|
| Security | Use strong authentication, role-based access, encryption in transit and at rest, signed credentials, and secure key management. |
| Privacy | Keep personal data off the public ledger; apply consent, purpose limitation, and minimum disclosure. |
| Scalability | Scale API, verification, notification, and integration services horizontally as users and credentials increase. |
| Performance | Use indexed credential lookup, caching for public issuer keys, and asynchronous events for non-critical work. |
| Reliability | Use replicated services, durable queues, backups, retries, and idempotent processing. |
| Availability | Deploy critical services redundantly and monitor service health and dependencies. |
| Maintainability | Separate modules, version APIs and schemas, automate testing, and document interfaces. |
| Interoperability | Support standards-based credential formats and APIs for LMS and HR platforms. |
| Auditability | Maintain tamper-evident records of credential lifecycle and verification events. |

## 3.2 Constraints and assumptions

The platform assumes that issuers are onboarded and their identities are checked before they receive signing authority. Learners have access to a device and can authenticate to their accounts. External HR and learning platforms use approved API credentials. The system depends on secure key custody and reliable issuer governance.

A decentralized ledger does not automatically guarantee that the original learning claim is true. Trust begins with the issuer's assessment process. The platform proves the origin and integrity of a credential and exposes its current status; it does not independently prove that every course assessment was valid.

## 3.3 Quality attributes influencing architecture

Security and privacy influence where data is stored and how credentials are signed. Interoperability influences credential formats and API design. Availability and performance influence service replication, caching, and asynchronous processing. Maintainability influences modular service boundaries.

# 4. Selection of Suitable Software Architecture

## 4.1 Selected architecture: Hybrid layered, microservices, and event-driven architecture

A hybrid architecture is selected. The presentation and API layers provide controlled access; domain services implement credential workflows; a message broker distributes lifecycle events; and a permissioned ledger or tamper-evident registry stores integrity proofs and status references.

The architecture is layered because user interfaces, APIs, business rules, and data persistence have distinct responsibilities. It uses service-oriented modules so credential issuance, portfolio management, verification, and integration can be developed and scaled independently. Event-driven communication supports notifications, audit updates, and integration without forcing every operation to wait for downstream services.

## 4.2 Why the architecture is suitable

First, independent services can scale according to workload. Verification traffic may increase during recruitment periods, while issuance traffic may follow academic calendars. Second, event-driven processing decouples notifications and analytics from the main credential workflow. Third, a ledger-backed integrity layer supports tamper evidence and shared verification across organizations.

Fourth, API boundaries simplify integration with HR systems and learning platforms. Fifth, the modular structure supports future credential standards and new issuer types. Finally, privacy controls can be enforced in the application and identity layers while only hashes, identifiers, and status proofs are anchored to the ledger.

## 4.3 Alternative architectures considered

| Architecture | Strength | Limitation for this project |
|---|---|---|
| Monolithic | Simple initial deployment and development. | Tightly coupled modules make independent scaling and integration harder as the platform grows. |
| Layered only | Clear separation of presentation, logic, and data. | Does not by itself provide asynchronous cross-organization event handling. |
| Pure public blockchain | Shared verification without one central operator. | Public exposure, transaction cost, privacy, and governance may be unsuitable for learner records. |
| Serverless only | Convenient for event-triggered tasks and variable demand. | Vendor dependence and execution limits can complicate long-running workflows and portability. |
| Hybrid selected design | Combines modular services, controlled data access, and tamper-evident proofs. | Requires governance, key management, monitoring, and distributed-system operations. |

## 5. Structural Architecture Diagram

### 5.1 Overall logical architecture

The following editable architecture is represented as a Word table so that each component label can be changed directly. Arrows indicate the main request, event, and verification paths.

| |
|---|
| **PRESENTATION LAYER**<br>Learner Portal \| Issuer Console \| Recruiter Verification Portal \| Admin Console |
| **ACCESS & IDENTITY LAYER**<br>API Gateway \| OAuth2 / OpenID Connect \| MFA \| Role-Based Access Control |
| **APPLICATION SERVICES**<br>Credential Issuance \| Portfolio Manager \| Sharing & Consent \| Verification \| Issuer Registry |
| **INTEGRATION & EVENT LAYER**<br>REST APIs \| Webhooks \| Message Broker \| HR / ATS Connector \| LMS Connector \| Notifications |
| **TRUST & CRYPTOGRAPHY LAYER**<br>Digital Signature Service \| Hashing \| Credential Status / Revocation \| Permissioned Ledger |
| **DATA MANAGEMENT LAYER**<br>Encrypted Credential Store \| Learner Profile DB \| Issuer Key Registry \| Audit Log \| Backup / Archive |
| **EXTERNAL ACTORS & SYSTEMS**<br>Learners \| Universities \| Online Course Platforms \| Employers / Recruiters \| HR and LMS Systems |

**↓ Data / request / event flow ↓**

Main flow: an issuer authenticates and submits credential data; the issuance service validates the issuer and signs the credential; the integrity proof is anchored to the ledger; the learner claims it into a portfolio; the learner grants scoped access; and a recruiter verifies signature, hash, issuer, and status.

### 5.2 Data placement

Personal details, full credential documents, and supporting evidence are stored in encrypted off-chain databases or object storage. The ledger stores a credential identifier, cryptographic hash, issuer reference, timestamp, and status pointer. This reduces unnecessary disclosure and limits ledger growth.

### 5.3 Trust boundaries

External issuers and HR systems connect through authenticated APIs. The API gateway validates identity and permissions. The signing service protects issuer keys. The data layer is accessible only to authorized services. The ledger interface accepts validated transactions from approved services.

## 6. Editable End-to-End Workflow Diagram

### 6.1 Credential lifecycle workflow

| |
|---|
| **1. ISSUER ONBOARDING**<br>Institution / platform submits registration → administrator verifies organization and authorized issuer identity. |
| **2. CREDENTIAL CREATION**<br>Issuer enters learner, skill, course, evidence, issue date, and optional expiry details. |
| **3. VALIDATION & SIGNING**<br>System checks required fields and issuer authority → credential is digitally signed → hash is generated. |
| **4. REGISTRY ANCHORING**<br>Credential ID, hash, issuer reference, timestamp, and status are recorded in the tamper-evident registry. |
| **5. LEARNER CLAIM**<br>Learner receives notification, authenticates, reviews the credential, and adds it to the portfolio. |
| **6. CONTROLLED SHARING**<br>Learner selects credentials, recipient, purpose, and expiry → system creates a limited verification link or QR code. |
| **7. RECRUITER VERIFICATION**<br>Recruiter opens link or API → platform checks signature, hash, issuer trust, status, and expiry. |
| **8. RESULT & AUDIT**<br>Verification result is displayed → access and verification event are logged → learner may revoke the share permission. |

**↓ Data / request / event flow ↓**

The workflow is editable: each numbered stage is a separate table row and can be moved, renamed, or expanded in Word. The sequence supports the complete credential lifecycle from trusted issuance to controlled verification.

### 6.2 Exception paths

If an issuer is not approved, issuance is rejected. If required fields are missing, the credential is returned for correction. If a signature or hash check fails, verification returns an invalid result. If a credential is revoked or expired, the verifier receives the current status rather than a valid result. If a sharing token expires, the learner must create a new permission.

If an external HR platform is unavailable, the event can be queued and retried. The platform must avoid duplicate issuance or duplicate audit records by using unique request identifiers and idempotent processing.

## 7. Explain Components and Their Interactions

### 7.1 Learner portal

The learner portal allows users to view their credentials, organize them into a portfolio, review issuer information, and control sharing permissions. It communicates with backend services through secured APIs and does not directly access the ledger or database.

### 7.2 Issuer management and credential issuance service

The issuer registry stores approved organization details, issuer roles, and public-key references. The issuance service validates that the requesting organization is authorized to issue the specified credential type. It checks mandatory fields, generates a credential identifier, signs the credential, and sends the integrity proof to the registry.

### 7.3 Portfolio and credential storage service

The portfolio service associates claimed credentials with a learner account and supports search, grouping, and display. The storage service keeps full credential data and supporting evidence encrypted. Access is restricted by learner ownership and consent settings.

### 7.4 Sharing and consent service

This service creates scoped access grants. A grant identifies which credential can be viewed, by whom, for what purpose, and until when. It can be revoked by the learner. Short-lived tokens and access logs reduce the risk of uncontrolled redistribution.

### 7.5 Verification service

The verification service checks the digital signature against the trusted issuer key, recomputes the credential hash, checks the registry record, and confirms that the credential has not expired or been revoked. It returns a clear status such as valid, invalid, revoked, expired, or issuer-untrusted.

### 7.6 Ledger and cryptographic service

The cryptographic service creates hashes and digital signatures. The ledger interface writes integrity proofs and status changes to the tamper-evident registry. Private keys must be protected using a managed key vault or hardware security module where available. Key rotation and issuer revocation are part of the governance process.

# 8. Integration, Data Management, and Security

## 8.1 API and external integration

REST APIs support account administration, credential issuance, portfolio retrieval, and verification. Webhooks notify external systems about issuance or status changes. HR/ATS integrations can request verification during recruitment, while LMS integrations can trigger credential issuance after course completion.

Each integration uses a registered client identity, scoped permissions, request validation, and rate limits. API versions and credential schemas must be documented to prevent incompatible updates.

## 8.2 Data model overview

| Entity | Important fields / relationship |
|---|---|
| Learner | Learner ID, verified contact, profile settings; owns a portfolio. |
| Issuer | Issuer ID, organization name, approval status, authorized users, public key reference. |
| Credential | Credential ID, learner ID, issuer ID, skill, course, evidence reference, issue date, expiry, signature, status. |
| Portfolio | Portfolio ID, learner ID, credential references, categories and display preferences. |
| Share Grant | Grant ID, learner ID, credential scope, recipient, purpose, token, expiry, revocation state. |
| Verification Event | Event ID, credential ID, verifier, timestamp, result, request reference. |
| Audit Event | Actor, action, object ID, timestamp, outcome, correlation ID. |

## 8.3 Security controls

Use TLS for data in transit and strong encryption for stored personal information. Require multi-factor authentication for privileged issuer and administrator accounts. Apply least-privilege access and role-based authorization. Protect signing keys, rotate credentials, validate API inputs, and monitor suspicious access patterns.

The ledger should not contain personal data or full certificates. A hash is a one-way integrity fingerprint, but it may still be linkable in some contexts; therefore identifiers and metadata should be minimized and access to verification records should be governed.

## 9. Discuss Quality Attributes

### 9.1 Scalability

The API gateway, verification service, notification service, and integration workers can be deployed as multiple stateless instances behind load balancers. Message queues buffer bursts in credential issuance and verification. Database indexes and read replicas support frequent portfolio and verification lookups.

### 9.2 Security and privacy

Digital signatures prove that a credential was signed by a trusted issuer and hashes detect content changes. Authentication and authorization protect user and issuer functions. Consent-based sharing and off-chain personal data storage reduce unnecessary disclosure. Security testing and key-management procedures remain essential.

### 9.3 Performance

Credential verification should use efficient hash comparison, cached issuer public keys, indexed status lookups, and minimal synchronous dependencies. Notifications, analytics, and audit exports can run asynchronously. Performance must be measured under realistic peak loads rather than assumed.

### 9.4 Reliability

Durable queues, retries with limits, idempotency keys, database backups, and replicated services help recover from component failures. The system should preserve an audit trail when downstream integrations are temporarily unavailable.

### 9.5 Maintainability

Separate services and documented APIs allow teams to update issuance, verification, portfolio, and integration functions independently. Automated unit, integration, and security tests reduce regression risk. Schema versioning supports gradual migration.

### 9.6 Availability

Critical APIs and databases should use redundant deployment, health checks, monitoring, and tested recovery procedures. If the ledger is temporarily unavailable, the system may queue anchoring requests but must clearly distinguish pending proofs from confirmed registry records.

## 10. Comparison Table: Architectural Alternatives

| Criteria | Monolithic | Layered only | Public blockchain only | Selected hybrid design |
|---|---|---|---|---|
| Deployment | One application | One application arranged in layers | Smart contracts plus clients | Independent services and shared trust registry |
| Scalability | Scale entire application | Usually scale application as a whole | Depends on network throughput | Scale busy services independently |
| Integration | Custom point-to-point modules | API boundaries possible | External adapters required | Versioned APIs, webhooks, and event broker |
| Privacy | Central database controls | Central database controls | Public data visibility risk | Personal data off-chain; minimal proof on registry |
| Tamper evidence | Requires protected logs | Requires protected logs | Strong shared ledger properties | Ledger proof plus signed credential |
| Maintainability | Can become tightly coupled | Clear internal separation | Contract upgrades can be complex | Modular services and versioned interfaces |
| Main trade-off | Simple start, harder growth | Good organization, limited event decoupling | Transparency versus privacy and governance | More operational complexity, better balance |

The hybrid design is preferred because the system needs both controlled personal-data management and shared integrity verification. A ledger-only solution would not provide the required portfolio, consent, and integration experience by itself.

# 11. Justification of Architectural Decisions

## 11.1 Decision rationale

Microservice boundaries follow business capabilities: issuer management, credential issuance, learner portfolio, sharing, verification, and integration. This avoids placing all functions in one tightly coupled deployment.

Event-driven communication is used for notifications, audit events, and external integration because these tasks can be processed asynchronously. Synchronous APIs remain appropriate for user login, portfolio retrieval, and immediate verification responses.

The permissioned ledger or tamper-evident registry is used to record integrity proofs and status events. Full learner data remains off-chain to support privacy, correction, retention, and access-control requirements.

## 11.2 Trade-offs

| Benefit | Trade-off / mitigation |
|---|---|
| Independent scaling | Requires service orchestration and distributed monitoring; use centralized logs, metrics, and tracing. |
| Tamper-evident proofs | Ledger governance and transaction costs must be managed; store only necessary proof data. |
| Selective sharing | Consent flows add user steps; provide clear permissions and simple expiry controls. |
| Digital signatures | Key loss or compromise can affect trust; use secure key storage, rotation, and revocation. |
| Event-driven integration | Events may be delayed or duplicated; use idempotency, retries, and dead-letter queues. |

## 11.3 Future enhancements

Future versions may support additional credential standards, cross-border issuer trust frameworks, skills analytics, AI-assisted skill mapping, verifiable presentation wallets, and stronger privacy-preserving proofs. These should be introduced through versioned APIs and modular services.

# 12. Technology Stack and Implementation Plan

## 12.1 Suggested technology stack

| Component | Possible technology | Purpose |
|---|---|---|
| Frontend | React.js | Learner, issuer, recruiter, and admin interfaces. |
| Backend APIs | Python FastAPI or Java Spring Boot | Business logic and secured endpoints. |
| Identity | OAuth 2.0 / OpenID Connect provider | Authentication and delegated authorization. |
| Credential format | W3C Verifiable Credentials-compatible model | Portable, structured credential representation. |
| Cryptography | Approved digital-signature library and key vault | Credential signing and verification. |
| Ledger | Permissioned blockchain or append-only tamper-evident registry | Store hashes, issuer references, and status events. |
| Database | PostgreSQL | Profiles, credential metadata, consent, and issuer data. |
| Object storage | Encrypted S3-compatible storage | Supporting documents and evidence. |
| Messaging | RabbitMQ or Apache Kafka | Asynchronous events and integration. |
| Monitoring | Prometheus and Grafana | Metrics, service health, and alerts. |

## 12.2 Implementation phases

Phase 1 – Requirements and data model: finalize actors, credential fields, trust rules, consent requirements, and acceptance criteria.

Phase 2 – Identity and issuer onboarding: implement learner accounts, issuer approval, roles, and authentication.

Phase 3 – Issuance and portfolio: create signed credentials, store them securely, and allow learners to claim and organize them.

Phase 4 – Ledger anchoring and verification: generate hashes, record proofs, and implement signature/status verification.

Phase 5 – Sharing and integrations: add scoped links, QR verification, HR/ATS APIs, LMS events, and notifications.

Phase 6 – Testing and deployment: perform functional, security, privacy, load, recovery, and interoperability tests.

# 13. Testing Strategy and Risk Analysis

## 13.1 Testing strategy

| Test ID | Scenario | Expected result |
|---|---|---|
| TC01 | Approved issuer creates a complete credential. | Credential is signed and assigned a unique ID. |
| TC02 | Unapproved user attempts issuance. | Request is rejected and security event is logged. |
| TC03 | Learner claims an issued credential. | Credential appears in the correct portfolio. |
| TC04 | Credential content is modified after signing. | Hash/signature validation fails. |
| TC05 | Recruiter verifies a valid credential. | Valid status and trusted issuer are returned. |
| TC06 | Credential has been revoked. | Verifier receives revoked status. |
| TC07 | Share token is expired. | Access is denied and learner can create a new grant. |
| TC08 | HR integration endpoint is unavailable. | Event is queued and retried without duplicate processing. |
| TC09 | High concurrent verification load. | Throughput and latency are measured against defined targets. |
| TC10 | Database or service failure occurs. | Recovery procedure restores service and preserves records. |

## 13.2 Risk analysis

| Risk | Impact | Mitigation |
|---|---|---|
| Issuer key compromise | Fraudulent credentials may be signed. | Secure key vault, rotation, incident revocation, and audit. |
| Incorrect issuer onboarding | Untrusted credentials enter the system. | Organization verification and periodic re-approval. |
| Personal data leakage | Learner privacy is affected. | Off-chain encrypted storage, access control, and data minimization. |
| Ledger/network outage | Proof anchoring or verification may be delayed. | Queue transactions, monitor status, and provide clear pending state. |
| Duplicate events | Duplicate credentials or audit entries. | Idempotency keys and unique transaction identifiers. |
| Weak integration security | Unauthorized HR or LMS access. | Client authentication, scoped tokens, rate limits, and logging. |

## 14. Evaluation According to the Given Rubric

The proposed report addresses the five assessment categories through requirement analysis, architecture selection, structural diagrams, component explanations, quality-attribute evaluation, and technical justification.

| Evaluation criterion | Weightage | Evidence in this report |
|---|---|---|
| System requirement analysis | 30% | Sections 2 and 3: objectives, functional and non-functional requirements, constraints. |
| Selection of software architecture | 25% | Section 4: hybrid architecture and comparison with alternatives. |
| Architecture diagram and component design | 20% | Sections 5, 6, and 7: structural architecture, workflow, and component responsibilities. |
| Component interaction and quality attributes | 15% | Sections 6, 7, and 9: lifecycle workflow and six quality attributes. |
| Architectural justification and technical presentation | 10% | Sections 10–13: comparison, trade-offs, technology stack, implementation, testing, and risks. |

To achieve the highest rubric level, the final implementation should demonstrate readable diagrams, complete component responsibilities, explicit interfaces, and measured test evidence. Performance values should be reported only after the prototype has been tested.

# 15. Conclusion and References

## 15.1 Conclusion

The proposed Decentralized Lifelong Learning Ledger provides a secure and portable approach to managing micro-credentials across educational institutions, online platforms, training providers, and employers. It allows trusted issuers to create digitally signed credentials, learners to maintain a lifelong portfolio, and recruiters to verify achievements through controlled access.

The hybrid layered, microservices, and event-driven architecture separates user interfaces, identity, credential workflows, integration, cryptographic validation, and data management. A tamper-evident registry supports integrity verification, while personal information and detailed documents remain in protected off-chain storage.

The design addresses scalability, security, privacy, performance, reliability, maintainability, and availability through independent services, encryption, role-based access, event processing, backups, and monitoring. It also recognizes important trade-offs, including ledger governance, key management, and distributed-system complexity.

In conclusion, the platform can reduce manual verification effort and improve learner mobility when implemented with trusted issuer governance, interoperable credential formats, secure key management, and rigorous testing. The architecture provides a flexible foundation for future integrations and evolving workforce skill requirements.

## 15.2 References

1. W3C, Verifiable Credentials Data Model v2.0, W3C Recommendation, 2025. https://www.w3.org/TR/vc-data-model-2.0/

2. W3C, Verifiable Credentials Overview. https://www.w3.org/TR/vc-overview/

3. NIST, Digital Identity Guidelines, Special Publication 800-63. https://pages.nist.gov/800-63-3/

4. NIST, Secure Software Development Framework (SSDF), SP 800-218. https://csrc.nist.gov/pubs/sp/800/218/final

5. ISO/IEC 25010, Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model.

6. OWASP, Application Security Verification Standard (ASVS). https://owasp.org/www-project-application-security-verification-standard/

7. Hyperledger Foundation, Documentation and resources for permissioned distributed ledger technologies. https://www.hyperledger.org/

8. Sommerville, I., Software Engineering, Pearson, software architecture and requirements engineering chapters.